Security of ML Systems
Adversarial Machine Learning: Attacks

Lecture Goals

- Systems approach
  - Training-time defense: “Adversarial Training”
  - Post-training defense: “Feature Squeezing”
- Certified Defense

Adversarial Training

Core idea

- If NN are “universal” approximators, then make them learn also adversarial examples!
- The approach is largely a brute-force one; the defender generates a large number of adversarial examples to train the model on, ensuring that it classifies both the original data point x and its perturbations in the same way
- In this type of training, the model can be trained directly using either the original data or the perturbed data, or analtered loss function can be used that also takes the perturbed examples into account

Adversarial training objective

- Why do they work? Because models are trained under **Empirical risk minimization** (ERM)
- Unfortunately, ERM often does not yield models that are robust to adversarially crafted examples.
- Can we define the attack? For example:
  - For each data point x, we introduce a set of allowed perturbations $S \subseteq R^d$ that formalizes the manipulative power of the adversary.
  - In image classification, we choose S so that it captures perceptual similarity between images. For instance, we saw the l_∞-ball around x
- We can modify the definition of the population risk $E_D[L]$ by incorporating the adversary described above. Rather than evaluating the loss L directly on samples drawn from the distribution D, the adversary is first allowed to perturb the input.
- This leads to the following formulation:
- $\min_{\theta} \quad \rho(\theta), \textit{where } \rho(\theta) = \mathbb{E}_{(x,y)\sim D}\left[\max_{\delta \in S} \quad L(\theta, x + \delta, y)\right].$

Madry et al. https://arxiv.org/pdf/1706.06083 Goodfellow *et al*. http://arxiv.org/abs/1412.6572

Adversarial training objective

- Under the perturbation in the l_∞-ball around x the following cost function (based on the **FGSM attack**) can be proposed to train the model:
  - $\tilde{L}_y(x, t) = \alpha L_y(x, t) + (1-\alpha) L_y\left(x + \epsilon \cdot \text{sign}\left(\nabla_x L_y(x, t)\right), t\right)$
  - where L_y is the loss function and t is the correct class of the point x.
- Under the PGD (Projected Gradient Descent) the attacker performs **multiple steps**:
  - $x_{t+1} = \Pi_{x+S}\left(x_t + \alpha\, sign\left(\nabla_x L(\theta, x_t, y)\right)\right)$
  - Process:
    - start from x
    - repeatedly increase the loss via gradient steps
    - **project back into the allowed perturbation set S**

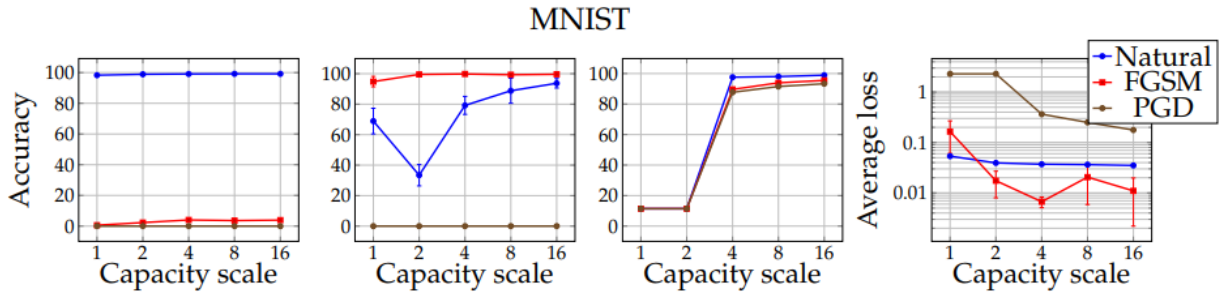
Madry et al. https://arxiv.org/pdf/1706.06083 Goodfellow *et al*. http://arxiv.org/abs/1412.6572

Adversarial training results

- MNIST dataset:
  - a simple convolutional network (a convolutional layer with 2 filters, followed by another convolutional layer with 4 filters, and a fully connected hidden layer with 64 units)
    - study how its behavior changes against different adversaries as we keep doubling the size of network (i.e. double the number of convolutional filters and the size of the fully connected layer)
    - adversarial examples are constructed with $\varepsilon = 0.3$.
- CIFAR10 dataset:
  - ResNet model
  - To increase the capacity, we modified the network incorporating wider layers by a factor of 10.
  - Adversarial examples were constructed with $\varepsilon = 8$

Madry et al. https://arxiv.org/pdf/1706.06083

Adversarial training results



CIFAR10

	Simple	Wide	Simple	Wide	Simple	Wide	Simple	Wide
Natural	92.7%	95.2%	87.4%	90.3%	79.4%	87.3%	0.00357	0.00371
FGSM	27.5%	32.7%	90.9%	95.1%	51.7%	56.1%	0.0115	0.00557
PGD	0.8%	3.5%	0.0%	0.0%	43.7%	45.8%	1.11	0.0218
(a) Standard training			(b) FGSM training		(c) PGD training		(d) Training Loss	

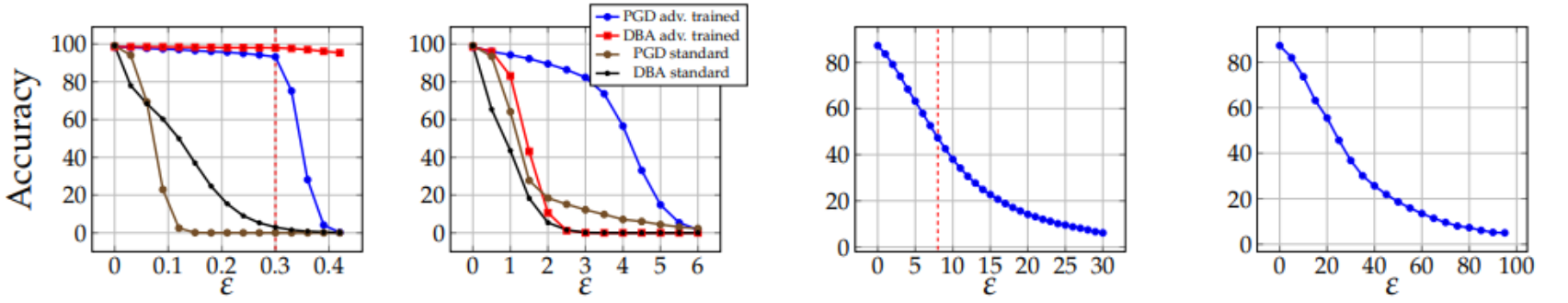
Figure 4: The effect of network capacity on the performance of the network. We trained MNIST and CIFAR10 networks of varying capacity on: (a) natural examples, (b) with FGSM-made adversarial examples, (c) with PGD-made adversarial examples. In the first three plots/tables of each dataset, we show how the standard and adversarial accuracy changes with respect to capacity for each training regime. In the final plot/table, we show the value of the cross-entropy loss on the adversarial examples the networks were trained on. This corresponds to the value of our saddle point formulation (2.1) for different sets of allowed perturbations.

- In MNIST, AT reduces an error rate from 89.4% to 17.9% on FGSM
- AT with FGSM don't increase the robustness to strong attacks…
- … unless one scale also the model capacity

Madry et al. https://arxiv.org/pdf/1706.06083

Adversarial training discussion

- Does it works? Not in all cases



(a) MNIST, ℓ_∞-norm (b) MNIST, ℓ_2-norm (c) CIFAR10, ℓ_∞-norm (d) CIFAR10, ℓ_2-norm

Figure 6: Performance of our adversarially trained networks against PGD adversaries of different strength. The MNIST and CIFAR10 networks were trained against $\varepsilon = 0.3$ and $\varepsilon = 8$ PGD ℓ_∞ adversaries respectively (the training ε is denoted with a red dashed lines in the ℓ_∞ plots). In the case of the MNIST adversarially trained networks, we also evaluate the performance of the Decision Boundary Attack (DBA) [4] with 2000 steps and PGD on standard and adversarially trained models. We observe that for ε less or equal to the value used during training, the performance is equal or better. For MNIST there is a sharp drop shortly after. Moreover, we observe that the performance of PGD on the MNIST ℓ_2-trained networks is poor and significantly overestimates the robustness of the model. This is potentially due to the threshold filters learned by the model masking the loss gradients (the decision-based attack does not utilize gradients).

Madry et al. https://arxiv.org/pdf/1706.06083

Defensive Distillation

- The concept of **Model Distillation** was first introduced as a way to transfer knowledge from a very large neural network to a smaller one.
- Let a neural network y trained on dataset D be given, and assume that the final prediction is obtained by applying the Softmax function to the logit layer, yielding a probability distribution over the classes.
- The distillation process consists of training a new network y' using the dataset D, where instead of the original target, the prediction $y(X)$ is used. In this way, more information about the different classes is available compared to having only a label corresponding to the most probable class.
- In addition, the final layer is modified as follows:
- $y_i(x) = \frac{e^{Z_i(x)/T}}{\sum_{j=1}^{|Y|} e^{Z_j(x)/T}}$
- where T is a distillation parameter called the temperature, Y is the set of classes, and Z is the output of the logit layer.
- It has been experimentally shown that higher values of T improve the performance of distillation. The use of this function makes the loss function smoother, leading to better generalization on unseen datasets and improved accuracy even on adversarial examples.

Papernot et al., https://arxiv.org/pdf/1511.04508

Defensive Distillation

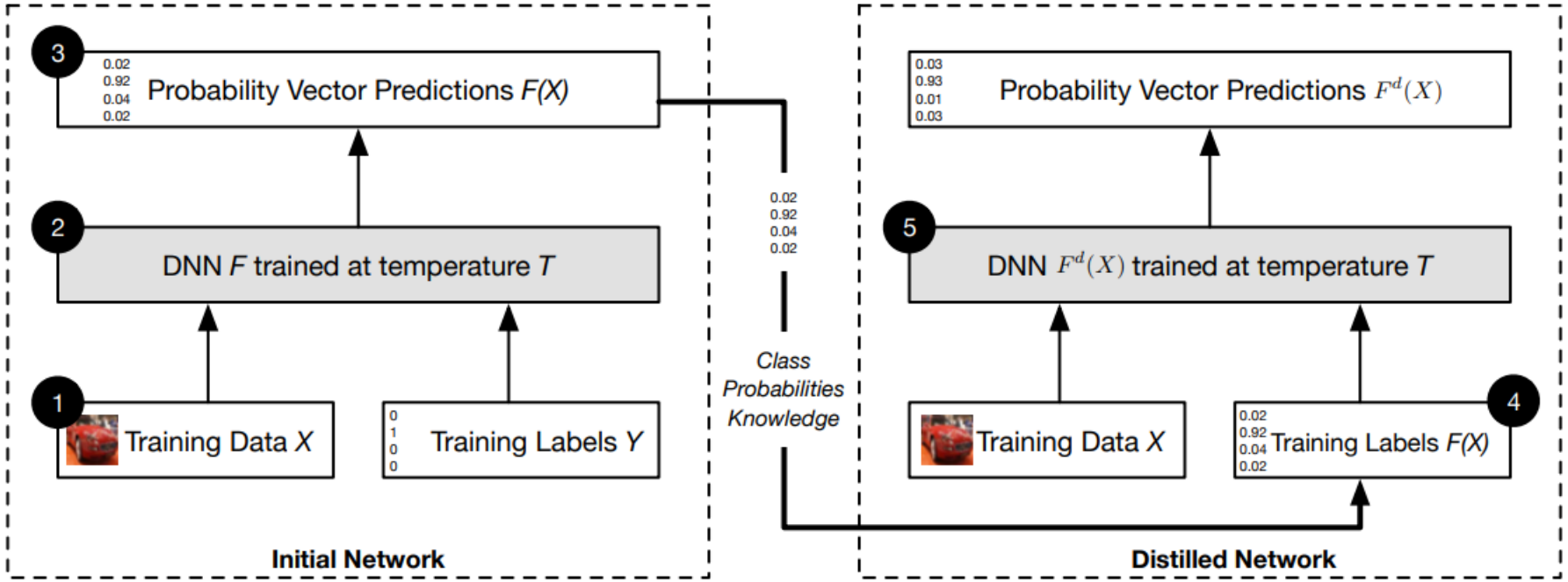


Fig. 5: **An overview of our defense mechanism based on a transfer of knowledge contained in probability vectors through distillation:** We first train an initial network F on data X with a softmax temperature of T. We then use the probability vector $F(X)$, which includes additional knowledge about classes compared to a class label, predicted by network F to train a distilled network F^d at temperature T on the same data X.

Papernot et al., https://arxiv.org/pdf/1511.04508

Defensive Distillation results

- Q: Does defensive distillation improve resilience against adversarial samples while retaining classification accuracy?
- Result: Distillation reduces the success rate of adversarial crafting from 95.89% to 0.45% on our first DNN and dataset, and from 87.89% to 5.11% on a second DNN and dataset. Distillation has negligible or non existent degradation in model classification accuracy in these settings.
- Is the end of the story?
  - We already saw the attacks that breaks it, it is the Carlini & Wagner

Papernot et al., https://arxiv.org/pdf/1511.04508

Defensive Distillation negative results

	Best Case				Average Case				Worst Case			
	MNIST		CIFAR		MNIST		CIFAR		MNIST		CIFAR	
	mean	prob	mean	prob	mean	prob	mean	prob	mean	prob	mean	prob
Our L_0	10	100%	7.4	100%	19	100%	15	100%	36	100%	29	100%
Our L_2	1.7	100%	0.36	100%	2.2	100%	0.60	100%	2.9	100%	0.92	100%
Our L_∞	0.14	100%	0.002	100%	0.18	100%	0.023	100%	0.25	100%	0.038	100%

- Average Case: select the target class uniformly at random among the labels that are not the correct label.
- Best Case: perform the attack against all incorrect classes, and report the target class that was least difficult to attack.
- Worst Case: perform the attack against all incorrect classes, and report the target class that was most difficult to attack.

Carlini & Wagner https://arxiv.org/pdf/1608.04644

Robustness against adversarial attacks seems a very difficult problem

Failed Defenses

Generative pretraining

Removing perturbation with an autoencoder

Adding noise at test time

Ensembles

Confidence-reducing perturbation at test time

Error correcting codes

Multiple glimpses

Weight decay

Double backprop

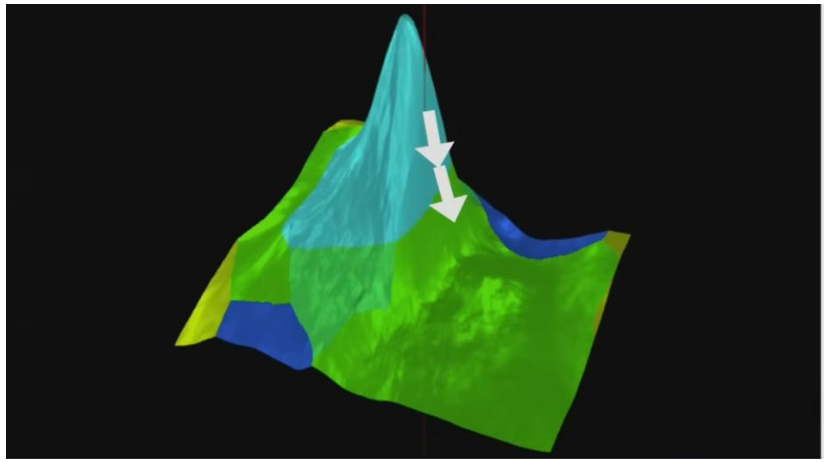
Adding noise at train time

Various non-linear units

Dropout

For a more clear point of view

- https://www.youtube.com/watch?v=umfeF0Dx-r4 (first 10mins)
- Very brief summary:
  - Easy to make ML models fail, they are vulnerable to perturbations
  - Based on GD



  - So a lot of defenses came out… and verifying the defense automatically with the GD. And they systematically failed

Robustness against adversarial attacks seems a very difficult problem

- A simple defense against gradient-based attacks consists of hiding gradient-related information from the adversary.
- This reduces the amount of information available and therefore forces the adversary to use less effective attacks.
- However, due to the property of transferability, this technique can be bypassed by creating a differentiable substitute model
- This also allows even non-differentiable models to be attacked using gradient-based methods.

Certified Defense based on DP

- Provide formal guarantees on the model's predictions within a specified perturbation budget
- guarantee: model's prediction remains unchanged within **the ϵ-neighborhood of x**

$$f(x) = f(x'), \quad \forall x' \in B(x, \epsilon)$$

input

Adversarial input

ϵ-neighbourhood of x: all possible inputs within distance ϵ from x

Lecuyer et al. https://arxiv.org/abs/1802.03471

Certified Defense based on DP

- A defense called **PixelDP**
- It injects **carefully calibrated random noise** into a neural network (DP-style noise)
- Then they use **differential privacy theory** to prove a bound on how much the model's output can change when the input is slightly perturbed
- This leads to a **certified robustness guarantee** of the form:
- *If an adversary perturbs the input within a bounded norm (e.g., small L_1 or L_2 changes), the prediction cannot change "too much" with high probability.*
  - **Robusteness:**
    - We must establish a definition for insensitivity to small changes to the input
    - We say a model f is insensitive, or robust, to attacks of p-norm L on a given input x if f(x) = f(x + α) for all $\alpha \in B_p(L)$. If f is a multiclass classification model based on label scores, this is equivalent to:

$$\forall \alpha \in B_p(L) \cdot y_k(x + \alpha) > \max_{i:i \neq k} y_i(x + \alpha)$$

Lecuyer et al. https://arxiv.org/abs/1802.03471

Certified Defense based on DP

Why DP connects to adversarial robustness

- The connection is conceptual:
- **Differential privacy guarantees:**
  - Changing one data point should not significantly change the output distribution.
- **Adversarial robustness goal**
  - Small changes in input should not change the prediction.
- So both are about **stability under small perturbations**.
- The paper exploits this analogy:
- “one data point change” (DP) ↔ “small input perturbation” (adversarial attack)
- That’s why DP machinery can be repurposed

Lecuyer et al. https://arxiv.org/abs/1802.03471

Certified Defense based on DP

- The approach relies on two key properties of DP:
  - **Post-processing property**: any computation applied to the output of an (ε, δ)-DP algorithm remains (ε, δ)-DP.
  - **Expected output stability property**: the expected value of an (ε, δ)- DP algorithm with bounded output is not sensitive to small changes in the input.

Lemma 1. (Expected Output Stability Bound) *Suppose a randomized function A, with bounded output $A(x) \in [0, b]$, $b \in \mathbb{R}^+$, satisfies (ϵ, δ)-DP. Then the expected value of its output meets the following property:*

$$\forall \alpha \in B_p(1) \,.\, \mathbb{E}(A(x)) \leq e^{\epsilon}\mathbb{E}(A(x+\alpha)) + b\delta.$$

The expectation is taken over the randomness in A.

Lecuyer et al. https://arxiv.org/abs/1802.03471

Certified Defense based on DP

- The intuition behind using DP to provide robustness to adversarial examples is to create a DP scoring function such that, given an input example, the predictions are DP with regards to the features of the input (e.g. the pixels of an image).
- In this setting, we can derive stability bounds for the expected output of the DP function using Lemma 1.
- The bounds, combined with our definition of robustness, give a rigorous condition (or certification) for robustness to adversarial examples.

Corollary 1. *Suppose a randomized function A satisfies (ϵ, δ)-PixelDP with respect to a p-norm metric, and where $A(x) = (y_1(x), \ldots, y_K(x)),\ y_k(x) \in [0, 1]$:*

$$\forall k, \forall \alpha \in B_p(1) \,.\, \mathbb{E}(y_k(x)) \leq e^{\epsilon}\mathbb{E}(y_k(x+\alpha)) + \delta. \quad (3)$$

Lecuyer et al. https://arxiv.org/abs/1802.03471

Certified Defense based on DP

- The approach is to transform a model's scoring function into a randomized (ε, δ)-PixelDP scoring function, A(x)
- Then the model's prediction procedure, f, use A's expected output over the DP noise, E(A(x)), as the label probability vector from which to pick the arg max

Proposition 1. (Robustness Condition) *Suppose A satisfies (ϵ, δ)-PixelDP with respect to a p-norm metric. For any input x, if for some $k \in \mathcal{K}$,*

$$\mathbb{E}(A_k(x)) > e^{2\epsilon} \max_{i:i \neq k} \mathbb{E}(A_i(x)) + (1 + e^{\epsilon})\delta, \qquad (4)$$

then the multiclass classification model based on label probability vector $y(x) = (\mathbb{E}(A_1(x)), \ldots, \mathbb{E}(A_K(x)))$ is robust to attacks α of size $\|\alpha\|_p \leq 1$ on input x.

Lecuyer et al. https://arxiv.org/abs/1802.03471

Certified Defense based on DP

Proposition 1. (Robustness Condition) *Suppose A satisfies (ϵ, δ)-PixelDP with respect to a p-norm metric. For any input x, if for some $k \in \mathcal{K}$,*

$$\mathbb{E}(A_k(x)) > e^{2\epsilon} \max_{i:i\neq k} \mathbb{E}(A_i(x)) + (1+e^{\epsilon})\delta, \qquad (4)$$

then the multiclass classification model based on label probability vector $y(x) = (\mathbb{E}(A_1(x)), \ldots, \mathbb{E}(A_K(x)))$ is robust to attacks α of size $\|\alpha\|_p \leq 1$ on input x.

Proof: Consider any $\alpha \in B_p(1)$, and let $x' := x + \alpha$. From Equation (3), we have:

$$\mathbb{E}(A_k(x)) \leq e^{\epsilon}\mathbb{E}(A_k(x')) + \delta, \qquad (a)$$
$$\mathbb{E}(A_i(x')) \leq e^{\epsilon}\mathbb{E}(A_i(x)) + \delta, \quad i \neq k. \qquad (b)$$

Equation (a) gives a lower-bound on $\mathbb{E}(A_k(x'))$; Equation (b) gives an upper-bound on $\max_{i\neq k} \mathbb{E}(A_i(x'))$. The hypothesis in the proposition statement (Equation (4)) implies that the lower-bound of the expected score for label k is strictly higher than the upper-bound for the expected score for any other label, which in turn implies the condition from Equation (1) for robustness at x. To spell it out:

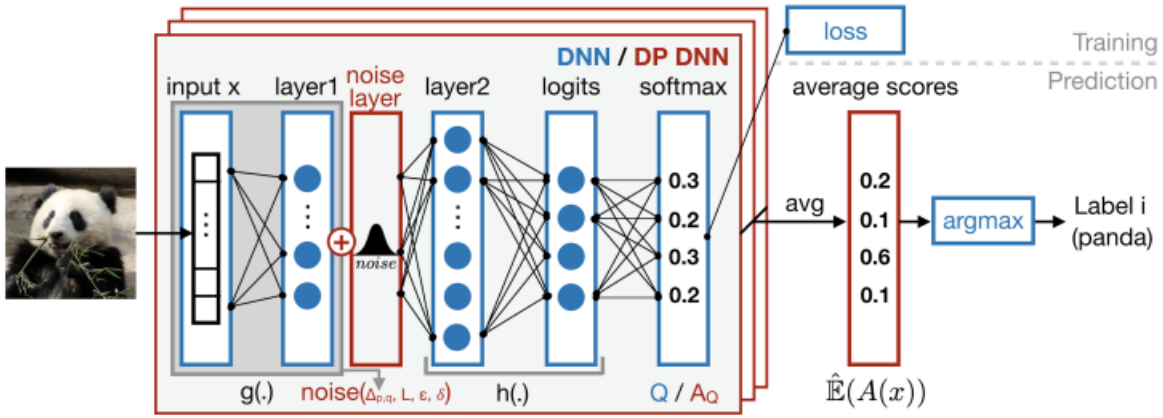
$$\begin{aligned}
\mathbb{E}(A_k(x')) &\overset{\text{Eq(a)}}{\geq} \frac{\mathbb{E}(A_k(x)) - \delta}{e^{\epsilon}} \\
&\overset{\text{Eq(4)}}{>} \frac{e^{2\epsilon} \max_{i:i\neq k} \mathbb{E}(A_i(x)) + (1+e^{\epsilon})\delta - \delta}{e^{\epsilon}} \\
&= e^{\epsilon} \max_{i:i\neq k} \mathbb{E}(A_i(x)) + \delta \\
&\overset{\text{Eq(b)}}{\geq} \max_{i:i\neq k} \mathbb{E}(A_i(x'))
\end{aligned}$$

$$\implies \mathbb{E}(A_k(x')) > \max_{i:i\neq k} \mathbb{E}(A_i(x+\alpha)) \ \forall \alpha \in B_p(1),$$

the very definition of robustness at x (Equation (1)). ■

Lecuyer et al. https://arxiv.org/abs/1802.03471

PixelDP

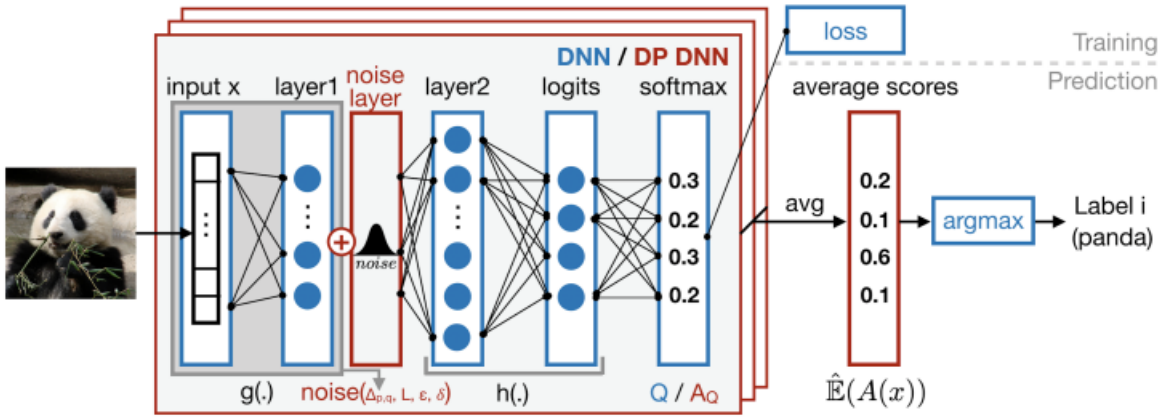


(a) PixelDP DNN Architecture

- PixelDP DNN architecture for multi-class image classification.
  - The original architecture is shown in blue; the changes introduced to make it PixelDP are shown in red.
  - Denote Q the original DNN's scoring function Q(x) = (y1(x), . . . , yK(x)).
- The vulnerability to adversarial examples stems from the unbounded sensitivity of Q with respect to p-norm changes in the input.
- Making the DNN (ε, δ)-PixelDP involves adding calibrated noise to turn Q into an (ε, δ)-DP randomized function A_Q; the expected output of that function will have bounded sensitivity to p-norm changes in the input.
- We achieve this by introducing a **noise layer** that adds zero mean noise to the output of the layer preceding it

Lecuyer et al. https://arxiv.org/abs/1802.03471

PixelDP



(a) PixelDP DNN Architecture

- Training an (ε, δ)-PixelDP network is similar to training the original network: we use the original loss and optimizer, such as stochastic gradient descent. The major difference is that we alter the pre-noise computation to constrain its sensitivity with regards to p-norm input changes.
- Prediction on the (ε, δ)-PixelDP scoring function, $A_Q(x)$, affords the robustness certification if the prediction procedure uses the expected scores, $E(A_Q(x))$, to select the winning label for any input x.

Lecuyer et al. https://arxiv.org/abs/1802.03471

PixelDP

Dataset	Baseline	$L = 0.03$	$L = 0.1$	$L = 0.3$	$L = 1.0$
ImageNet	77.5%	–	68.3%	57.7%	37.7%
CIFAR-10	95.5%	93.3%	87.0%	70.9%	44.3%
CIFAR-100	78.6%	73.4%	62.4%	44.3%	22.1%
SVHN	96.3%	96.1%	93.1%	79.6%	28.2%
MNIST	99.2%	99.1%	99.1%	98.2%	11%

Table III: **Impact of PixelDP noise on conventional accuracy.** For each DNN, we show different levels of construction attack size L. Conventional accuracy degrades with noise level.

Lecuyer et al. https://arxiv.org/abs/1802.03471

How about detecting incoming attacks?

- Make robust models:
  - (Gradient masking) Defensive distillation
  - Adversarial training
  - …
- Detect adversarial examples:
  - Sample statistics
  - Train a detector model
  - …

Information-theoretical Perspective

- What does it happen under compression?

- The feature input spaces are often unnecessarily large, and this vast input space provides extensive opportunities for an adversary to construct adversarial examples.

Lecuyer et al. https://arxiv.org/pdf/1704.01155

Main Idea: Feature Squeezing

- Reduce unnecessary input variability so that adversarial perturbations become detectable.
- **Transforming the input** using a “squeezer”
- **Running the model on both:**
  - original input
  - squeezed input
- **Comparing predictions**
- If the outputs differ significantly → the input is likely adversarial.
- This works because adversarial perturbations tend to be **unstable under small transformations**, while normal images are not.

Lecuyer et al. https://arxiv.org/pdf/1704.01155

Types of Feature Squeezing Used

- **1. Color Bit Depth Reduction** Reduce the number of colors per pixel.
  - Example:
    - 8-bit color (256 levels)
    - reduced to 4-bit (16 levels)
- This **removes small pixel-level perturbations** that attacks rely on.
- **2. Spatial Smoothing:** Apply a filter (e.g., median filter) to smooth the image.
- Effect: Removes high-frequency noise → Adversarial perturbations often disappear

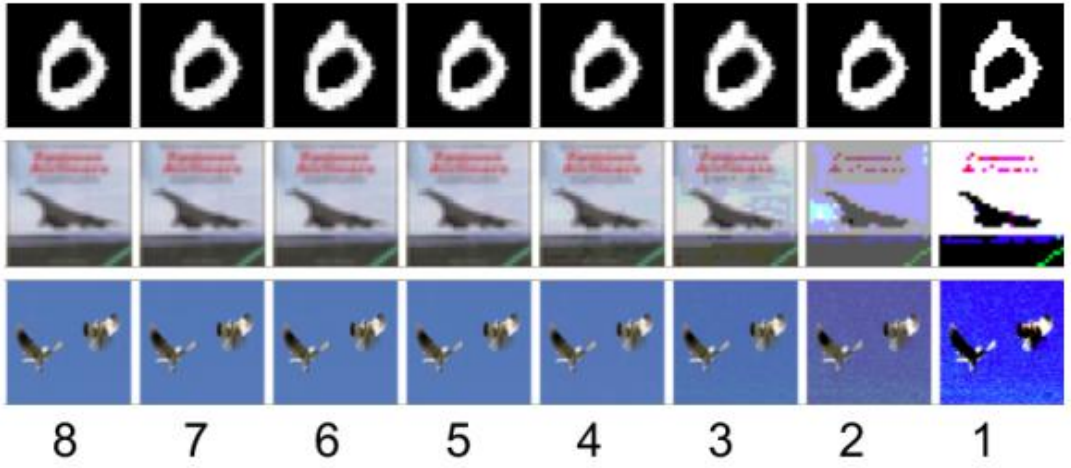
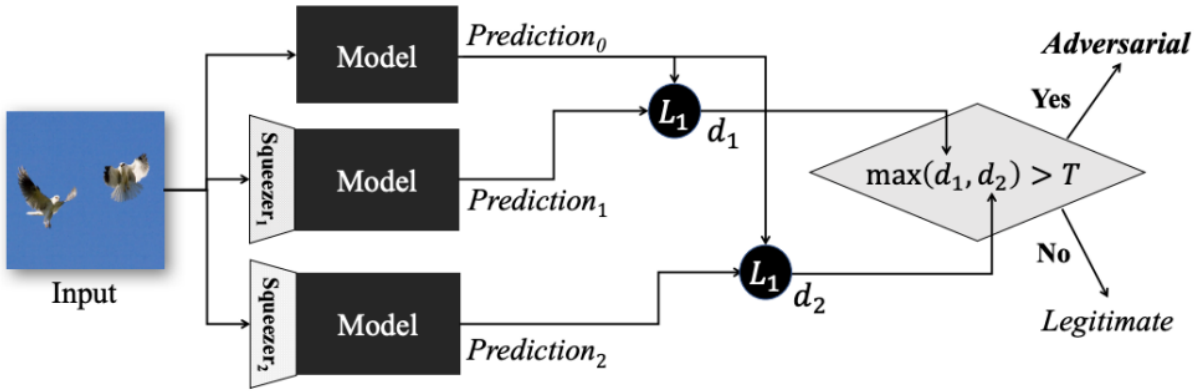


Fig. 2: Image examples with bit depth reduction. The first column shows images from MNIST, CIFAR-10 and ImageNet, respectively. Other columns show squeezed versions at different color-bit depths, ranging from 8 (original) to 1.

Lecuyer et al. https://arxiv.org/pdf/1704.01155

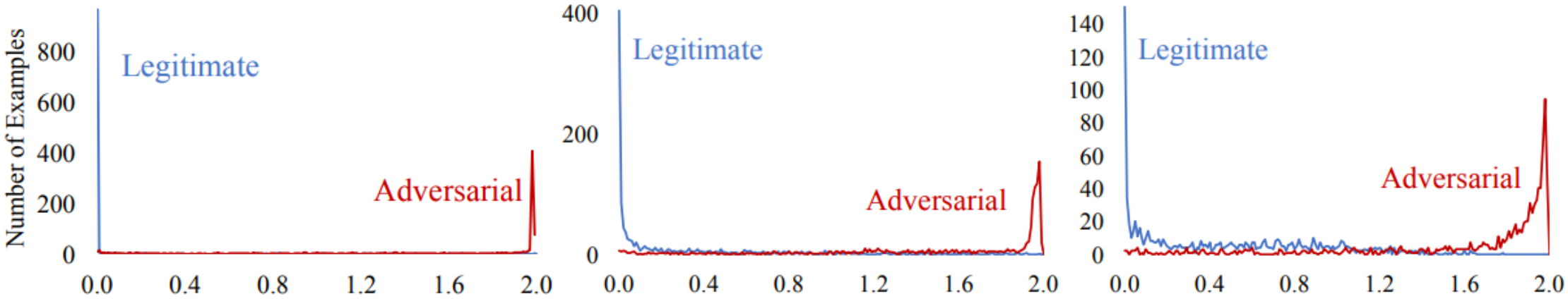
Detection Mechanism

- The detection score is:
  - difference = distance(model(original input), model(squeezed input))
  - If difference > threshold → adversarial.
- This method:
  - does not modify the model
  - only preprocesses inputs.



Lecuyer et al. https://arxiv.org/pdf/1704.01155

Detection Mechanism



(a) MNIST examples. (b) CIFAR-10 examples. (c) ImageNet examples.

Fig. 5: Differences in L_1 distance between original and squeezed sample, for legitimate and adversarial examples across three datasets. The L_1 score has a range from 0.0 to 2.0 . Each curve is fitted over 200 histogram bins each representing the L_1 distance range of 0.01. Each sample is counted in the bin for the maximum L_1 distance between the original prediction and the output of the best joint-detection squeezing configuration shown in Table 4. The curves for adversarial examples are for all adversarial examples, including unsuccessful ones (so the separation for successful ones is even larger than shown here).

Lecuyer et al. https://arxiv.org/pdf/1704.01155

Experimental Results

- Tested on: MNIST, CIFAR-10
- 7-layer CNN, DenseNet, and MobileNet
- 100 images correctly classified by them
- Against attacks such as: FGSM JSMA
- Results:
  - High adversarial detection rates
  - Low false positive rates
- Minimal computational overhead

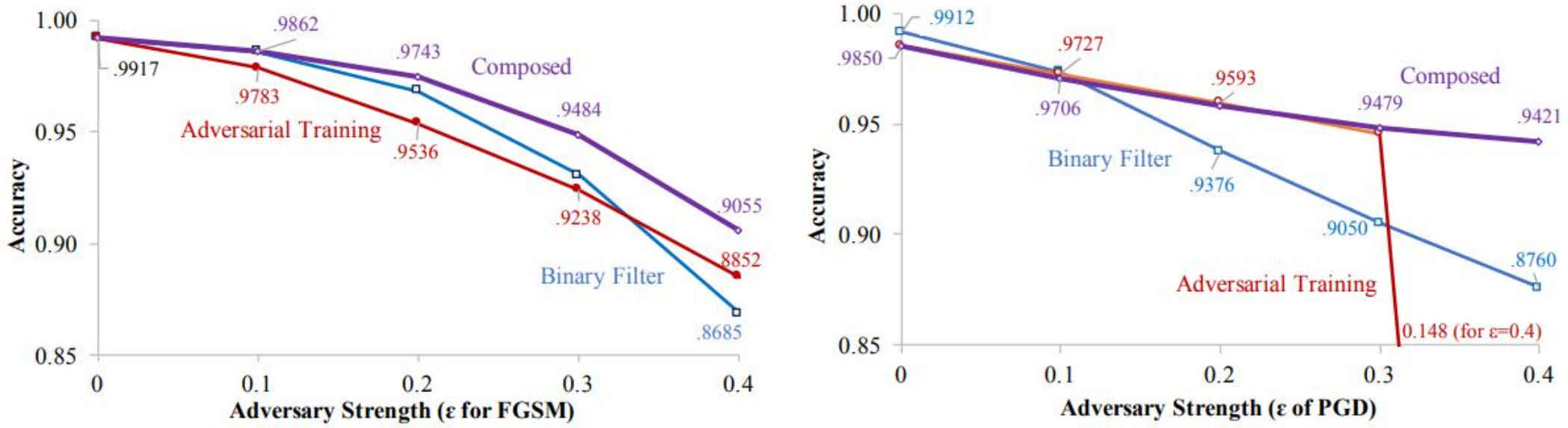
Lecuyer et al. https://arxiv.org/pdf/1704.01155

Model accuracy with feature squeezing

Dataset	Squeezer		L_∞ Attacks				L_2 Attacks			L_0 Attacks				All Attacks	Legitimate
	Name	Parameters	FGSM	BIM	CW_∞		Deep-Fool	CW_2		CW_0		JSMA			
					Next	LL		Next	LL	Next	LL	Next	LL		
MNIST	None		54%	9%	0%	0%	-	0%	0%	0%	0%	27%	40%	13.00%	99.43%
	Bit Depth	1-bit	**92%**	**87%**	**100%**	**100%**	-	**83%**	**66%**	0%	0%	50%	49%	**62.70%**	99.33%
	Median Smoothing	2x2	61%	16%	70%	55%	-	51%	35%	39%	36%	62%	56%	48.10%	99.28%
		3x3	59%	14%	43%	46%	-	51%	53%	**67%**	**59%**	**82%**	**79%**	55.30%	98.95%
CIFAR-10	None		15%	8%	0%	0%	2%	0%	0%	0%	0%	0%	0%	2.27%	94.84%
	Bit Depth	5-bit	17%	13%	12%	19%	40%	40%	47%	0%	0%	21%	17%	20.55%	94.55%
		4-bit	21%	29%	69%	74%	72%	84%	84%	7%	10%	23%	20%	44.82%	93.11%
	Median Smoothing	2x2	**38%**	**56%**	**84%**	**86%**	**83%**	**87%**	83%	**88%**	**85%**	**84%**	**76%**	**77.27%**	89.29%
	Non-local Means	11-3-4	27%	46%	80%	84%	76%	84%	**88%**	11%	11%	44%	32%	53.00%	91.18%
ImageNet	None		1%	0%	0%	0%	11%	10%	3%	0%	0%	-	-	2.78%	69.70%
	Bit Depth	4-bit	5%	4%	66%	79%	44%	84%	82%	38%	67%	-	-	52.11%	68.00%
		5-bit	2%	0%	33%	60%	21%	68%	66%	7%	18%	-	-	30.56%	69.40%
	Median Smoothing	2x2	22%	28%	75%	81%	**72%**	81%	84%	**85%**	**85%**	-	-	**68.11%**	65.40%
		3x3	**33%**	**41%**	73%	76%	66%	77%	79%	81%	79%	-	-	67.22%	62.10%
	Non-local Means	11-3-4	10%	25%	**77%**	**82%**	57%	**87%**	**86%**	43%	47%	-	-	57.11%	65.40%

Lecuyer et al. https://arxiv.org/pdf/1704.01155

Model accuracy with feature squeezing



(a) FGSM attacks.

(b) PGD attacks.

Fig. 4: Composing adversarial training with feature squeezing. The horizontal axis is the adversary's strength (ϵ), increasing to the right. The adversarial training uses $\epsilon = 0.3$ for both FGSM are PGD. Composing the 1-bit filter with the adversarial-trained model often performs the best.

Lecuyer et al. https://arxiv.org/pdf/1704.01155

Unfortunately

- https://arxiv.org/pdf/1803.09868

BYPASSING FEATURE SQUEEZING BY INCREASING ADVERSARY STRENGTH

Yash Sharma[1] and Pin-Yu Chen[2]

[1]The Cooper Union, New York, NY 10003, USA
[2]IBM Research, Yorktown Heights, NY 10598, USA
sharma2@cooper.edu, pin-yu.chen@ibm.com

Unfortunately

- One of the most widely observed patterns is:
- **Almost every proposed defense is eventually bypassed by a stronger or adaptive attack.**
- A classic example is the paper:
  - Adversarial Example Defenses: Ensembles of Weak Defenses Are Not Strong
  - It showed that combining several defense mechanisms **still fails against adaptive adversaries**, because attackers can optimize directly against the entire defense system

Unfortunately

- A theoretical paper explicitly explores whether adversarial examples are unavoidable:
- Are Adversarial Examples Inevitable?
- This work shows that **for certain problem classes and high-dimensional datasets, adversarial examples must exist** due to geometric properties of the data distribution.
- Key idea:
- In high-dimensional spaces (like images),
- decision boundaries are extremely complex,
- so small perturbations can cross them easily.
- **Implication:** Even a well-trained classifier may inherently have adversarial examples.

Unfortunately

- Another difficulty is that adversarial attacks are essentially **optimization problems**.
- Researchers note that:
- adversarial examples are solutions to **non-linear, non-convex optimization problems**
- predicting all such solutions is extremely difficult.
- Because of this:
- defenses often work only against **specific attack algorithms**
- new attack methods can bypass them.

Acknowledgments

- Sources cited in slides
- All papers discussed
- Some slides from:
  - https://trustworthy-ml-course.github.io/ Lecture 3
  - https://true-lab.ai/courses/MLSec/Sp23/syllabus.html Part II